



LeetCode 1207 – Unique Number of Occurrences

1. Problem Title & Link

Title: LeetCode 1207: Unique Number of Occurrences

Link: <https://leetcode.com/problems/unique-number-of-occurrences/>

2. Problem Statement (Short Summary)

Given an integer array arr, determine whether the number of occurrences of each value is unique.

Return:

true

if all frequencies are unique.

Otherwise return:

false

3. Examples (Input → Output)

Example 1

Input:

arr = [1,2,2,1,1,3]

Output:

true

Explanation:

Frequency:

1 → 3 times

2 → 2 times

3 → 1 time

Occurrences:

3, 2, 1

All unique.

Example 2

Input:

arr = [1,2]

Output:

false

Explanation:

1 → 1 time



2 → 1 time

Frequency:

1, 1

Not unique.

Example 3

Input:

arr = [-3,0,1,-3,1,1,1,-3,10,0]

Output:

true

4. Constraints

1 <= arr.length <= 1000

-1000 <= arr[i] <= 1000

Important:

Need uniqueness of frequencies,
not uniqueness of elements.

5. Core Concept (Pattern / Topic)

HashMap + HashSet

Frequency Counting Pattern.

6. Thought Process (Step-by-Step Explanation)

Brute Force Approach

Count frequency of each element.

Then compare every frequency with every other frequency.

Example:

1 → 3

2 → 2

3 → 1

Compare:

3 with 2



3 with 1

2 with 1

Time Complexity:

$O(k^2)$

where k = distinct elements.

Optimized Thinking

First calculate frequencies.

Example:

1 → 3

2 → 2

3 → 1

Store frequencies in a set.

Set

{3,2,1}

If:

Map Size == Set Size

then all frequencies are unique.

Otherwise duplicates exist.

7. Visual / Intuition Diagram (ASCII)

```
Input:
[1,2,2,1,1,3]
Frequency Map:
1 → 3

2 → 2

3 → 1
Convert frequencies to set:
{3,2,1}
Map size:
3
Set size:
3
Equal:
true
```



8. Pseudocode

```
create frequency map

for each number
    increase frequency

create set

add all frequencies

return

map size == set size
```

9. Code Implementation

Python

```
class Solution:
    def uniqueOccurrences(self, arr):

        freq = {}

        for num in arr:
            freq[num] = freq.get(num, 0) + 1

        return len(freq.values()) == len(set(freq.values()))
```

Java

```
import java.util.*;

class Solution {

    public boolean uniqueOccurrences(int[] arr) {

        HashMap<Integer,Integer> map = new HashMap<>();

        for(int num : arr) {

            map.put(num,
                    map.getOrDefault(num, 0) + 1);
        }

        HashSet<Integer> set = new HashSet<>();

        for(int count : map.values()) {
```



```
        set.add(count);  
    }  
  
    return map.size() == set.size();  
}  
}
```

10. Time & Space Complexity

Time Complexity

$O(n)$

One pass for counting.

One pass for frequencies.

Space Complexity

$O(n)$

Map + Set.

11. Common Mistakes / Edge Cases

Mistake 1

Checking element uniqueness.

Problem asks:

Frequency uniqueness

not value uniqueness.

Mistake 2

Using array length comparison.

Need:

Map Size

vs

Set Size

Mistake 3

Forgetting negative values are allowed.

12. Detailed Dry Run

Input:

[1,2,2,1,1,3]

Build Frequency Map



Number	Frequency
1	3
2	2
3	1

Build Set

Insert:

3

2

1

Set:

{1,2,3}

Size:

3

Map Size:

3

Result:

true

13. Common Use Cases (Real-Life / Interview)**Frequency Validation**

Examples:

Product sales counts

Student attendance counts

Log frequency analysis

Transaction occurrence checks

Determine whether counts themselves are unique.

14. Common Traps (Important!)**Trap 1**

Confusing values and frequencies.

Trap 2



Checking duplicates in original array.

Trap 3

Using nested loops.

HashSet simplifies the solution.

15. Builds To (Related LeetCode Problems)

LC 217 Contains Duplicate



LC 1512 Number of Good Pairs



LC 1207 Unique Number of Occurrences



LC 242 Valid Anagram



LC 49 Group Anagrams



LC 347 Top K Frequent Elements

HashMap progression.

16. Alternate Approaches + Comparison

Approach	Time	Space	Notes
Nested Frequency Comparison	$O(k^2)$	$O(k)$	Slow
Sort Frequencies	$O(k \log k)$	$O(k)$	Good
HashSet Frequencies	$O(n)$	$O(n)$	Optimal

Approach 1: Sort Frequencies

Example:

[3,2,1]

Sort and check neighbors.

Works well.

Approach 2: HashSet

Store frequencies.

If duplicate frequency exists:



Set size decreases

Best solution.

17. Why This Solution Works (Short Intuition)

A Set automatically removes duplicates.

If all frequencies are unique:

Map Size = Set Size

Otherwise:

Set Size < Map Size

Therefore the comparison directly tells us whether occurrence counts are unique.

18. Variations / Follow-Up Questions

Follow-Up 1

Return duplicate frequencies.

Follow-Up 2

Return elements having duplicate frequencies.

Follow-Up 3

Find the most frequent element.

Leads to:

LC 347 Top K Frequent Elements

Follow-Up 4

Check if exactly one frequency is duplicated.

Variation of the same pattern.